UNIVERSIDADE DE BRASÍLIA

ENGENHARIA DE COMPUTAÇÃO

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

PROJETO DO BANCO DE REGISTRADORES RISC-V

Implementação e verificação em VHDL — regs

Aluno: João Pedro de Souza Lopes

Matricula:221006342

Brasília — DF | Outubro de 2026

1. Introdução

O banco de registradores é um componente fundamental do caminho de dados de um processador RISC-V. Ele disponibiliza operandos para as operações da unidade de processamento e armazena resultados produzidos por instruções. Este projeto implementa um banco com 32 posições, duas portas de leitura e uma porta de escrita, com a particularidade de que o registrador x0 permanece constante em zero.

Segue em conjunto deste relatorio imagens simulacoes ,blocos de codigo entre outros apendices.

2. Objetivos

Projetar, descrever e verificar em VHDL um banco de registradores semelhante ao do RISC-V, com palavras de 32 bits, escrita síncrona na borda de subida do relógio, leituras assíncronas e proteção do registrador x0. Elaborar um testbench que preencha o banco, confira as duas saídas de leitura por meio de assert e tente escrever um valor não nulo em x0.

3. Especificação e interface

Sinal	Função
clk	Relógio; escrita na borda de subida.
wren	Habilitação de escrita.
rd	Endereço de escrita (5 bits).
data	Dado de escrita (WSIZE bits).
rs1, rs2	Endereços independentes de leitura (5 bits).
ro1, ro2	Saídas independentes de leitura (WSIZE bits).
WSIZE	Largura parametrizável da palavra; padrão 32 bits.

4. Implementação

O banco é representado por um tipo array com 32 elementos do tipo std_logic_vector. Um processo sensível ao clock realiza a escrita somente na borda de subida, quando wren = 1 e rd ≠ 0. Duas atribuições concorrentes implementam as leituras assíncronas. A inicialização do array em zero foi utilizada para facilitar a observação em simulação; em hardware, a disponibilidade de inicialização depende da tecnologia de síntese.

4.1. Tratamento do registrador x0

A constante zero é assegurada por dois mecanismos: a escrita em rd = 0 é bloqueada, e as saídas ro1 e ro2 são forçadas a zero quando rs1 ou rs2 selecionam o endereço 0. Assim, uma tentativa de escrita não modifica o comportamento arquitetural de x0.

5. Metodologia de verificação

O testbench gera clock de período 10 ns e executa oito etapas. Primeiro verifica a leitura de x0; depois escreve os valores i × 10 em x1 até x31. Em seguida, lê cada posição por ro1 e por ro2, comparando com os valores esperados por assert. Por fim, testa uma escrita não nula em x0, uma tentativa de escrita com wren desabilitado, a sobrescrita de x5 e uma leitura simultânea de x5 e x10.

Etapa	Estímulo	Resultado esperado
1	Leitura de x0 em ambas as portas	ro1 = ro2 = 0
2	Escrita em x1…x31	xi = i × 10
3	Leitura sequencial por ro1	Valores previamente escritos
4	Leitura sequencial por ro2	Valores previamente escritos
5	Escrita FFFFFFFF em x0	x0 continua em zero
6	Escrita em x5 com wren = 0	x5 permanece 00000032
7	Sobrescrita de x5 com 12345678	Ambas as portas leem 12345678
8	Leitura simultânea x5 e x10	ro1 = 12345678; ro2 = 00000064

6. Formas de onda e resultados

As figuras abaixo devem ser preenchidas com capturas reais do GTKWave. Os resultados descritos nesta seção são os esperados pelo testbench; não foram fornecidos o arquivo VCD nem o log da execução para confirmar os resultados observados.

Escrita sequencial nos registradores

Sinais sugeridos: clk, wren, rd, data e etapa. Espera-se alteração de cada registrador apenas nas bordas de subida com wren = 1.

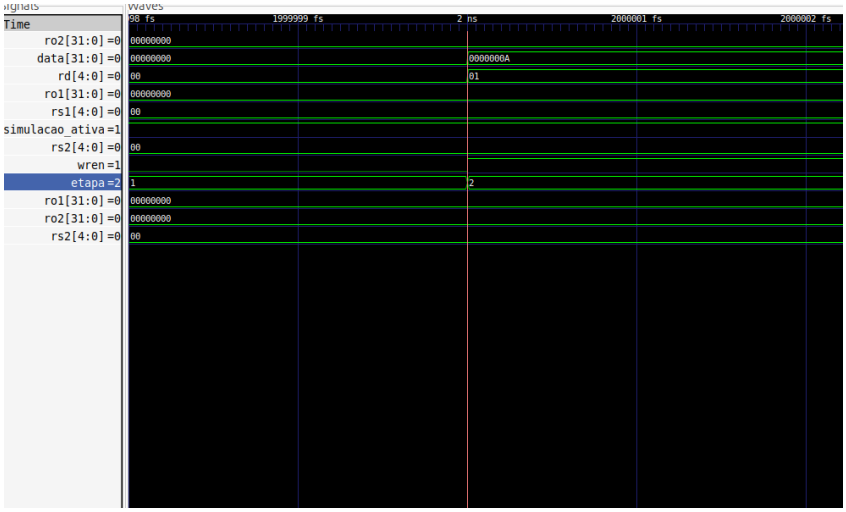


Figura — Escrita sequencial nos registradores. Fonte: simulação do projeto (a inserir após execução).

Leituras pelas portas ro1 e ro2

Sinais sugeridos: rs1, rs2, ro1, ro2 e etapa. Espera-se que cada saída corresponda ao endereço selecionado, sem depender de nova borda de clock.



Figura — Leituras pelas portas ro1 e ro2. Fonte: simulação do projeto (a inserir após execução).

Proteção do registrador x0

Sinais sugeridos: clk, wren, rd, data, rs1, rs2, ro1 e ro2. Mesmo com data = FFFFFFFF e rd = 0, as saídas devem permanecer em zero.

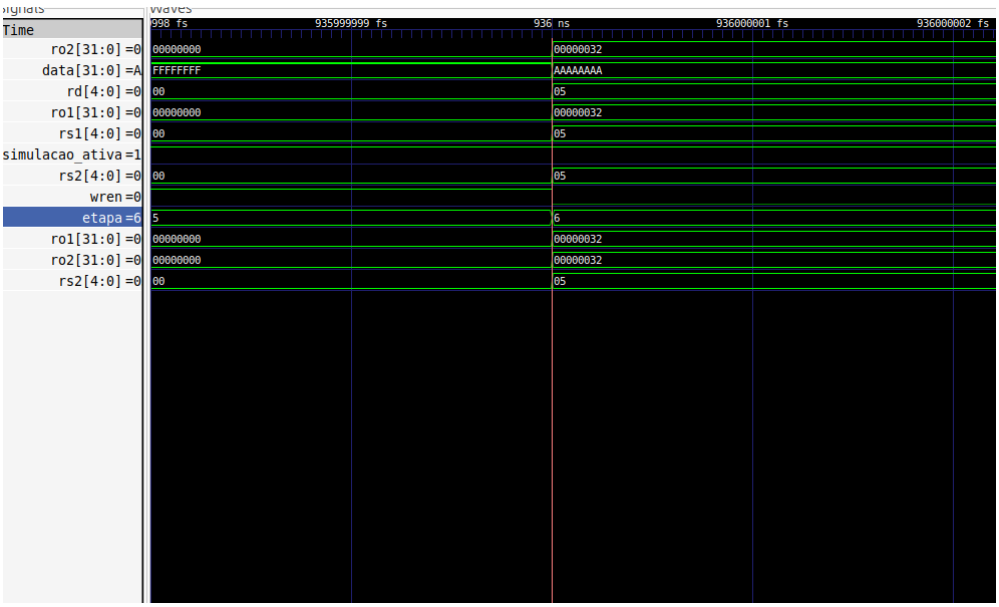


Figura — Proteção do registrador x0. Fonte: simulação do projeto (a inserir após execução).

Controle de escrita e leitura simultânea

Sinais sugeridos: clk, wren, rd, data, rs1, rs2, ro1, ro2. Espera-se bloqueio da escrita com wren = 0, atualização de x5 com wren = 1 e leitura independente de x5 e x10.

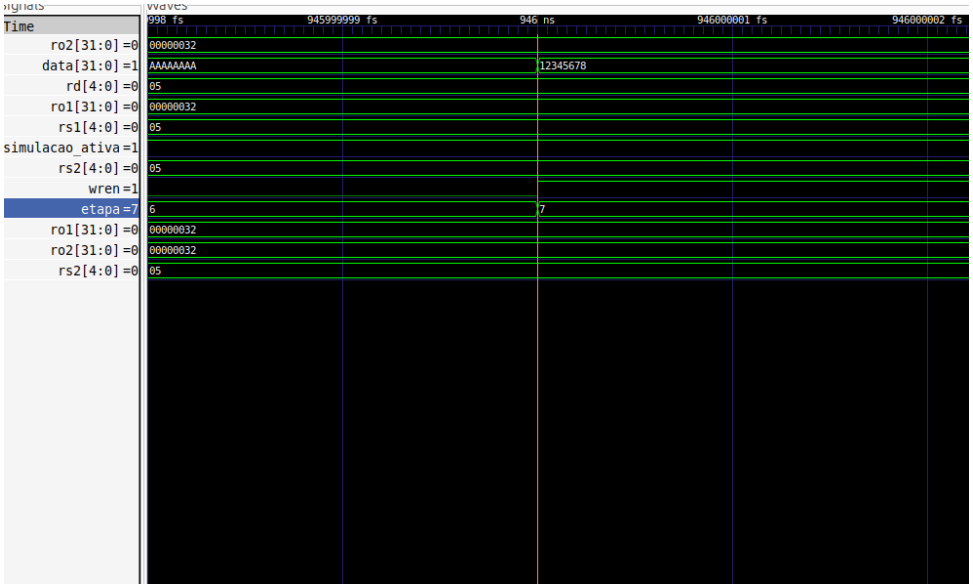


Figura — Controle de escrita e leitura simultânea. Fonte: simulação do projeto (a inserir após execução).

6.1. Interpretação dos resultados

A ausência de mensagens assert com severidade error, caso confirmada no log, indicará que os estímulos aplicados produziram os valores esperados. A visualização das ondas permitirá relacionar as bordas de subida à escrita e as mudanças de endereços às leituras assíncronas. O teste de x0 é especialmente importante, pois comprova que a interface preserva o valor zero mesmo diante de uma tentativa explícita de alteração.

7. Conclusão

Foi descrito um banco de registradores parametrizável, com 32 posições, duas portas de leitura e uma porta de escrita. A arquitetura adota escrita síncrona e leitura assíncrona, além de bloquear escritas em x0 e retornar zero quando esse registrador é selecionado. O testbench cobre o preenchimento do banco, a leitura pelas duas portas e situações adicionais de controle. A validação definitiva deve ser registrada após a execução da simulação e a inclusão das formas de onda reais.

Referência

UNIVERSIDADE DE BRASÍLIA. Arquitetura e Organização de Computadores. Projeto do Banco de Registradores do RISC-V. Enunciado da atividade, s.d.

Apêndice A — Código VHDL de regs.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;

entity regs is
    generic (WSIZE : natural := 32);
    port (
        clk, wren : in std_logic;
        rs1, rs2, rd : in std_logic_vector(4 downto 0);
        data : in std_logic_vector(WSIZE-1 downto 0);
        ro1, ro2 : out std_logic_vector(WSIZE-1 downto 0)
    );
end entity regs;

architecture comportamento of regs is
    type banco_registradores is array (0 to 31)
        of std_logic_vector(WSIZE-1 downto 0);
    signal registradores : banco_registradores :=
        (others => (others => '0'));
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if wren = '1' then
                if unsigned(rd) /= 0 then
                    registradores(to_integer(unsigned(rd))) <= data;
                end if;
            end if;
        end if;
    end process;

    ro1 <= (others => '0') when unsigned(rs1) = 0
           else registradores(to_integer(unsigned(rs1)));
    ro2 <= (others => '0') when unsigned(rs2) = 0
           else registradores(to_integer(unsigned(rs2)));
end architecture comportamento;
```

Apêndice B — Código VHDL de regs_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity regs_tb is
end entity regs_tb;

architecture teste of regs_tb is
    constant WSIZE : natural := 32;
    constant CLK_PERIOD : time := 10 ns;
    signal clk : std_logic := '0';
    signal wren : std_logic := '0';
    signal rs1, rs2, rd : std_logic_vector(4 downto 0) :=
        (others => '0');
    signal data : std_logic_vector(WSIZE-1 downto 0) :=
        (others => '0');
    signal ro1, ro2 : std_logic_vector(WSIZE-1 downto 0);
    signal etapa : integer := 0;
    signal simulacao_ativa : boolean := true;
begin
    DUT: entity work.regs
        generic map (WSIZE => WSIZE)
        port map (
            clk => clk, wren => wren,
            rs1 => rs1, rs2 => rs2, rd => rd,
            data => data, ro1 => ro1, ro2 => ro2
        );

    clock_process: process
    begin
        while simulacao_ativa loop
            clk <= '0';
            wait for CLK_PERIOD / 2;
            clk <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;
```

```
    stimulus: process
        variable esperado : std_logic_vector(WSIZE-1 downto 0);
    begin
        etapa <= 1;
        rs1 <= "00000"; rs2 <= "00000";
        wait for 2 ns;
        assert ro1 = x"00000000" and ro2 = x"00000000"
            report "Erro: x0 nao e zero" severity error;


        etapa <= 2;
        wren <= '1';
        for i in 1 to 31 loop
            rd <= std_logic_vector(to_unsigned(i, 5));
            data <= std_logic_vector(to_unsigned(i*10, WSIZE));
            wait until rising_edge(clk);
            wait for 1 ns;
        end loop;
        wren <= '0';


        etapa <= 3;
        for i in 1 to 31 loop
            rs1 <= std_logic_vector(to_unsigned(i, 5));
            esperado := std_logic_vector(to_unsigned(i*10, WSIZE));
            wait for 10 ns;
            assert ro1 = esperado
                report "Erro: leitura ro1" severity error;
        end loop;


        etapa <= 4;
        for i in 1 to 31 loop
            rs2 <= std_logic_vector(to_unsigned(i, 5));
            esperado := std_logic_vector(to_unsigned(i*10, WSIZE));
            wait for 10 ns;
            assert ro2 = esperado
                report "Erro: leitura ro2" severity error;
        end loop;


        etapa <= 5;
        rd <= "00000"; data <= x"FFFFFFFF"; wren <= '1';
        rs1 <= "00000"; rs2 <= "00000";
        wait until rising_edge(clk);
        wait for 1 ns;
        assert ro1 = x"00000000" and ro2 = x"00000000"
            report "Erro: escrita em x0" severity error;
        wren <= '0';


        etapa <= 6;
        rd <= "00101"; data <= x"AAAAAAAA"; wren <= '0';
        rs1 <= "00101"; rs2 <= "00101";
        wait until rising_edge(clk);
        wait for 1 ns;
        assert ro1 = x"00000032" and ro2 = x"00000032"
            report "Erro: escrita com wren=0" severity error;


        etapa <= 7;
        rd <= "00101"; data <= x"12345678"; wren <= '1';
        wait until rising_edge(clk);
        wait for 1 ns;
        assert ro1 = x"12345678" and ro2 = x"12345678"
            report "Erro: sobrescrita x5" severity error;
        wren <= '0';


        etapa <= 8;
        rs1 <= "00101"; rs2 <= "01010";
        wait for 10 ns;
        assert ro1 = x"12345678" and ro2 = x"00000064"
            report "Erro: leitura simultanea" severity error;


        etapa <= 9;
        report "Todos os testes foram executados" severity note;
        simulacao_ativa <= false;
        wait;
    end process;
end architecture teste;
```